

DOKUMEN PERANCANGAN PERANGKAT LUNAK (Software Design Document / SDD)

APLIKASI UMKM PENJUALAN SAYURAN SEGAR

Berbasis RESTful API (Golang + PostgreSQL + React.js)

Nomor Dokumen	SDD-UMKM-003
Versi	1.0
Tanggal	April 2026
Status	Final Draft
Berdasarkan SKPL	SKPL-UMKM-003, Maret 2026

Kelompok 3

M. Irvan Alfiansyah (20231310046) | M. Nur Yanfa (20231310047) | Cindy Oktaviani
(20231310044)

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA
Jln. Terusan Halimun No. 37, Bandung, Jawa Barat 40263

BAB I — PENDAHULUAN

1.1 Tujuan Dokumen

Dokumen Software Design Document (SDD) ini merupakan terjemahan teknis dari Spesifikasi Kebutuhan Perangkat Lunak (SKPL-UMKM-003) ke dalam cetak biru perancangan yang dapat langsung diimplementasikan oleh tim pengembang. Dokumen ini merinci keputusan arsitektur, desain antarmuka antar-komponen, skema basis data fisik, serta kontrak API yang menjadi pedoman implementasi backend Golang dan frontend React.js.

SDD ini berperan sebagai jembatan antara 'apa yang harus dibangun' (SKPL) dengan 'bagaimana cara membangunnya' (SDD), sehingga seluruh pemangku kepentingan — mulai dari engineer, QA, hingga pemimpin proyek — memiliki satu sumber kebenaran teknis yang sama.

1.2 Ruang Lingkup Sistem

Sistem yang dirancang adalah Aplikasi E-Commerce UMKM Sayuran Segar berbasis Decoupled Architecture yang terdiri dari:

- Backend RESTful API — dibangun dengan bahasa pemrograman Go (Golang) v1.20+, Gin Web Framework, dan GORM ORM.
- Basis Data Relasional — PostgreSQL v15 sebagai penyimpanan utama dengan jaminan ACID.
- Frontend SPA — React.js sebagai antarmuka pengguna yang berkomunikasi melalui HTTP/HTTPS ke backend API.
- Integrasi Pihak Ketiga — Midtrans Payment Gateway (Snap API + Webhook) untuk pemrosesan pembayaran, dan JNE/Sicepat untuk layanan logistik.

1.3 Daftar Singkatan dan Istilah

Singkatan/Istilah	Definisi
SDD	Software Design Document – dokumen perancangan perangkat lunak ini
SKPL	Spesifikasi Kebutuhan Perangkat Lunak – dokumen referensi (SKPL-UMKM-003)
API	Application Programming Interface – kontrak komunikasi antar layanan berbasis HTTP/JSON
JWT	JSON Web Token – token autentikasi stateless (RFC 7519)
ACID	Atomicity, Consistency, Isolation, Durability – sifat transaksi basis data
IDOR	Insecure Direct Object Reference – kerentanan akses objek tidak sah
RBAC	Role-Based Access Control – kendali akses berbasis peran pengguna
ORM	Object-Relational Mapping – pustaka pemetaan objek ke tabel (GORM)
SPA	Single Page Application – pola aplikasi web berbasis React.js
VPC	Virtual Private Cloud – jaringan privat isolasi untuk database server
Webhook	HTTP Push Notification dari Midtrans ke server saat transaksi selesai

1.4 Referensi

1. SKPL-UMKM-003, Aplikasi UMKM Penjualan Sayuran Segar, Kelompok 3, Maret 2026
2. IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions
3. Dokumentasi Resmi Go (Golang) v1.20 — <https://go.dev/doc/>
4. Dokumentasi Gin Web Framework — <https://gin-gonic.com/docs/>
5. Dokumentasi GORM ORM — <https://gorm.io/docs/>
6. Dokumentasi PostgreSQL v15 — <https://www.postgresql.org/docs/15/>
7. Dokumentasi Midtrans Snap API — <https://docs.midtrans.com/>
8. OWASP Top 10 Security Risks — <https://owasp.org/Top10/>
9. Robert C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017

BAB II — ARSITEKTUR SISTEM

2.1 Gambaran Arsitektur Global

Sistem mengadopsi Decoupled (Headless) Architecture di mana lapisan presentasi (frontend React.js) dan lapisan logika bisnis (backend Golang) sepenuhnya terpisah dan hanya berkomunikasi melalui kontrak API. Keputusan arsitektur ini memberikan manfaat: (1) skalabilitas independen, (2) kemudahan pengujian unit dan integrasi, (3) fleksibilitas penggantian frontend tanpa menyentuh backend.

Pada sisi backend, diterapkan pola Clean Architecture dengan empat lapisan isolasi:

- Domain Layer — entitas bisnis murni dan interface kontrak, tidak bergantung pada framework apapun.
- Repository Layer — implementasi akses data menggunakan GORM, mengabstraksi detail SQL dari logika bisnis.
- Usecase Layer — orkestrator logika bisnis; mengelola transaksi, validasi, dan aturan domain.
- Delivery Layer — HTTP Handler berbasis Gin yang bertugas parsing request, memanggil usecase, dan memformat response JSON.

2.2 Topologi Deployment

Infrastruktur sistem didistribusikan ke beberapa node komputasi dengan isolasi keamanan berlapis:

Komponen	Teknologi / Konfigurasi	Keterangan
Client Browser / Mobile	React.js SPA (HTTPS)	Merender antarmuka pengguna, konsumsi API
Admin Web Browser	React.js Admin Panel (HTTPS)	Dashboard pengelolaan platform
Web Server / Load Balancer	Nginx, Port 80/443	Terminasi SSL, reverse proxy ke App Server
Application Server	Golang Gin REST API, Port 8080, Linux OS	Eksekusi seluruh logika bisnis dan API
Database Server (Private VPC)	PostgreSQL v15, Port 5432	Penyimpanan data relasional, tidak dapat diakses publik
Object Storage	S3-compatible, HTTPS	Penyimpanan gambar produk dan dokumen
Payment Gateway	Midtrans Snap API + Webhook, HTTPS	Pemrosesan pembayaran eksternal
Shipping Service	JNE/Sicepat API, HTTPS	Kalkulasi ongkos kirim dan pelacakan

2.3 Pola Arsitektur Backend (Clean Architecture)

Setiap modul fungsional pada backend diorganisir mengikuti struktur direktori berikut:

Direktori	Layer	Tanggung Jawab
/cmd/api/main.go	Entry Point	Inisialisasi server, konfigurasi env, routing global
/internal/domain/*.go	Domain	Struct entitas & Interface kontrak Repository/Usecase
/internal/repository/*.go	Repository	Implementasi query GORM + transaksi database
/internal/usecase/*.go	Usecase	Logika bisnis, validasi, orkestrasi operasi
/internal/delivery/http/*.go	Delivery	HTTP Handler, binding JSON, response formatting
/internal/middleware/*.go	Middleware	Auth JWT, Role Check, Logger, Recovery, CORS
/pkg/jwt/jwt.go	Utility	Generate & Validate JWT token
/pkg/hash/bcrypt.go	Utility	Hash & Compare password dengan bcrypt cost 14
/config/*.go	Config	Baca environment variables (DB, JWT secret, Midtrans key)

2.4 Pola Komunikasi Antar Komponen

Alur komunikasi standar untuk setiap HTTP request yang masuk ke sistem mengikuti pola berikut:

- Client mengirim HTTP Request (JSON payload + JWT Bearer Token di header Authorization).
- Nginx menerima request, melakukan terminasi SSL, dan meneruskan ke Golang App Server via reverse proxy.
- Chain Middleware dieksekusi secara berurutan: LoggerMiddleware → RecoveryMiddleware → AuthMiddleware (validasi JWT) → RoleMiddleware (validasi peran).
- Delivery Layer (HTTP Handler) menerima request yang sudah terautentikasi, melakukan binding dan validasi JSON payload.
- Handler memanggil Usecase Layer dengan parameter yang sudah bersih.
- Usecase mengeksekusi logika bisnis, memanggil Repository Layer untuk operasi database.
- Repository Layer mengeksekusi query GORM ke PostgreSQL, mengembalikan domain object atau error.
- Usecase memproses hasil, Handler memformat response JSON, dan mengembalikan HTTP response ke client.

BAB III — DESAIN KOMPONEN DAN MODUL

3.1 Modul Autentikasi dan Manajemen Sesi (Auth Module)

3.1.1 Tanggung Jawab

Modul ini bertanggung jawab atas seluruh siklus identitas pengguna: pendaftaran akun baru, autentikasi, penerbitan token JWT, dan terminasi sesi. Modul ini menjadi gerbang utama sebelum pengguna dapat mengakses endpoint terproteksi manapun.

3.1.2 Interface Komponen

Method	Endpoint	Auth	Deskripsi
POST	/api/v1/auth/register	Publik	Registrasi akun pembeli baru
POST	/api/v1/auth/login	Publik	Login semua aktor, return JWT 24 jam
POST	/api/v1/auth/logout	JWT Required	Invalidasi token (stateful blacklist opsional)
GET	/api/v1/auth/me	JWT Required	Ambil profil pengguna aktif dari JWT

3.1.3 Algoritma Login

Algoritma login dirancang untuk memberikan keamanan optimal sambil tetap responsif:

18. Ekstrak dan validasi JSON payload {email, password} dari request body.
19. Eksekusi GORM query: `SELECT * FROM users WHERE email = ? LIMIT 1` (parameterized, mencegah SQL Injection).
20. Jika record tidak ditemukan, kembalikan HTTP 401 Unauthorized dengan pesan generik untuk mencegah user enumeration.
21. Eksekusi `bcrypt.CompareHashAndPassword(storedHash, inputPassword)` — komputasi sengaja lambat (~200-500ms) untuk melumpuhkan brute-force.
22. Jika bcrypt gagal, kembalikan HTTP 401 Unauthorized.
23. Bangun JWT Claims: {sub: id_user, role: role, exp: now+24h, iat: now}.
24. Tandatangani JWT dengan HMAC-SHA256 menggunakan `SERVER_JWT_SECRET` dari environment variable.
25. Kembalikan HTTP 200 OK dengan payload {token, role, expires_in: '24h'}.

3.1.4 Desain Middleware Auth

AuthMiddleware diinjeksikan ke semua route group yang memerlukan autentikasi:

26. Ekstrak header: `Authorization: Bearer <token>`.
27. Jika header tidak ada atau format salah, return 401 Unauthorized.
28. Verifikasi JWT signature menggunakan `SERVER_JWT_SECRET`.
29. Cek klaim exp — jika sudah kadaluarsa, return 401 dengan pesan 'token expired'.
30. Inject {id_user, role} ke dalam Gin Context untuk digunakan handler downstream.

3.2 Modul Manajemen Katalog Produk (Product Module)

3.2.1 Tanggung Jawab

Modul ini mengelola siklus hidup produk sayuran: penciptaan, pembacaan, pembaruan, dan penghapusan (CRUD) beserta perlindungan kepemilikan data (Ownership Protection / Anti-IDOR).

3.2.2 Interface Komponen

Method	Endpoint	Auth / Role	Deskripsi
GET	/api/v1/products	Publik	Daftar produk dengan filter, sort, pagination
GET	/api/v1/products/:id	Publik	Detail satu produk beserta stok terkini
POST	/api/v1/products	JWT: supplier	Tambah produk baru (id_user dari JWT)
PUT	/api/v1/products/:id	JWT: supplier	Update produk (validasi ownership IDOR)
DELETE	/api/v1/products/:id	JWT: supplier/admin	Hapus produk (validasi ownership / admin)
GET	/api/v1/products/categories	Publik	Daftar kategori produk

3.2.3 Algoritma Anti-IDOR Ownership Validation

Validasi kepemilikan wajib dieksekusi pada setiap operasi PUT dan DELETE:

31. Handler menerima id_produk dari URL parameter.
32. Ekstrak id_user dari Gin Context (disisipkan AuthMiddleware).
33. Eksekusi GORM query: `SELECT * FROM produk WHERE id_produk = ? AND id_user = ?`.
34. Klausula `AND id_user = ?` adalah kunci keamanan — query ini hanya sukses jika produk benar-benar milik supplier yang sedang login.
35. Jika hasil query nil (0 rows), return HTTP 403 Forbidden: 'Akses ditolak: Anda bukan pemilik produk ini'. Catat percobaan di `audit_log`.
36. Jika ditemukan, lanjutkan operasi UPDATE/DELETE.

3.3 Modul Keranjang Belanja (Cart Module)

3.3.1 Tanggung Jawab

Modul ini mengelola state keranjang belanja pembeli sebagai 'ruang tunggu' sebelum checkout. Operasi bersifat CRUD sederhana dengan validasi ketersediaan stok dasar.

3.3.2 Interface Komponen

Method	Endpoint	Auth / Role	Deskripsi
GET	/api/v1/shopping-carts	JWT: pembeli	Tampilkan isi keranjang pembeli aktif
POST	/api/v1/shopping-carts	JWT: pembeli	Tambah / update item ke keranjang
DELETE	/api/v1/shopping-carts/:id	JWT: pembeli	Hapus satu item dari keranjang

Method	Endpoint	Auth / Role	Deskripsi
DELETE	/api/v1/shopping-carts	JWT: pembeli	Kosongkan seluruh keranjang

3.4 Modul Pemesanan dan Transaksi ACID (Order Module)

3.4.1 Tanggung Jawab

Modul ini adalah inti transaksional sistem — mengubah intent pembelian (keranjang) menjadi komitmen bisnis (pesanan) dengan jaminan atomisitas penuh. Modul ini menangani skenario race condition pada pembelian bersamaan (concurrent checkout).

3.4.2 Interface Komponen

Method	Endpoint	Auth / Role	Deskripsi
POST	/api/v1/orders/checkout	JWT: pembeli	Proses checkout ACID, return snap_token
GET	/api/v1/orders	JWT: pembeli	Riwayat pesanan pembeli aktif
GET	/api/v1/orders/:id	JWT: pembeli/admin	Detail satu pesanan beserta status
PATCH	/api/v1/orders/:id/cancel	JWT: pembeli/admin	Batalkan pesanan (status PENDING saja)
GET	/api/v1/admin/orders	JWT: admin	Semua pesanan lintas pengguna (admin view)
PATCH	/api/v1/admin/orders/:id/process	JWT: admin	Update status ke PROCESSED

3.4.3 Algoritma Checkout ACID

Algoritma checkout dirancang untuk menjamin tidak ada overselling ($\text{stok} < 0$) bahkan dalam skenario ratusan concurrent user:

37. Autentikasi: Pastikan JWT valid dan role = pembeli.
38. Ambil isi keranjang: `SELECT * FROM keranjang WHERE id_user = ?`.
39. Validasi awal: Jika keranjang kosong, return HTTP 400 Bad Request.
40. Buka gerbang transaksi: `tx := db.Begin()` — semua operasi berikutnya dalam satu unit atomik.
41. Row-Level Locking per item: `SELECT stok FROM produk WHERE id_produk = ? FOR UPDATE` — kunci baris produk agar tidak bisa dibaca/dimodifikasi proses lain sampai transaksi commit/rollback.
42. Validasi stok: `IF stok < kuantitas THEN tx.Rollback() → return HTTP 400 {stok_tersedia, stok_diminta}`.
43. Kalkulasi total_tagihan: `SUM(harga * kuantitas)` untuk seluruh item.
44. Buat record pesanan: `INSERT INTO pesanan {id_user, total_tagihan, status='PENDING'}`.
45. Buat record item pesanan: `INSERT INTO order_items` untuk setiap item keranjang.

46. Kurangi stok: UPDATE produk SET stok = stok - kuantitas WHERE id_produk = ? — aman karena sudah ter-lock di langkah 5.
47. Request Midtrans Snap Token: POST ke <https://app.midtrans.com/snap/v1/transactions> dengan payload pesanan.
48. Jika Midtrans error: tx.Rollback() → return HTTP 502 Bad Gateway.
49. Simpan payment_token dan payment_url ke tabel orders.
50. Hapus isi keranjang: DELETE FROM keranjang WHERE id_user = ?.
51. Commit: tx.Commit() → return HTTP 201 Created dengan {id_pesanan, total_tagihan, snap_token, redirect_url}.

3.5 Modul Pembayaran (Payment Module)

3.5.1 Desain Webhook Handler

Endpoint webhook dirancang sebagai penerima notifikasi asinkronus dari Midtrans dan harus bersifat idempotent (dapat menerima notifikasi duplikat tanpa efek samping):

52. Terima HTTP POST dari Midtrans ke /api/v1/payments/webhook (tanpa autentikasi JWT — dilindungi signature validation).
53. Ekstrak payload: {order_id, status_code, gross_amount, signature_key}.
54. Rekonstruksi signature: hash_string = order_id + status_code + gross_amount + SERVER_KEY_MIDTRANS.
55. Hitung SHA-512: computed_signature = SHA512(hash_string).
56. Bandingkan: IF computed_signature ≠ signature_key → return HTTP 403 Forbidden. Log percobaan pemalsuan.
57. Cek idempotency: SELECT status FROM pesanan WHERE id_pesanan = ?. Jika sudah PAID, return HTTP 200 OK tanpa perubahan.
58. Jika status PENDING dan signature valid: UPDATE pesanan SET status='PAID', payment_method=?, paid_at=NOW().
59. Return HTTP 200 OK — Midtrans memerlukan 200 OK untuk menghentikan retry webhook.

3.5.2 Interface Komponen

Method	Endpoint	Auth	Deskripsi
POST	/api/v1/payments/webhook	Signature SHA-512	Terima notifikasi pembayaran dari Midtrans
GET	/api/v1/payments/:order_id/status	JWT: pembeli	Cek status pembayaran pesanan

3.6 Modul Pengiriman (Shipping Module)

3.6.1 Tanggung Jawab

Modul ini mengelola siklus hidup pengiriman fisik: penugasan kurir oleh admin, pembaruan status perjalanan barang oleh kurir, hingga konfirmasi penerimaan. Dilindungi validasi IDOR per-manifest.

3.6.2 Interface Komponen

Method	Endpoint	Auth / Role	Deskripsi
GET	/api/v1/shipping/:id	JWT: pembeli/kurir	Lacak status pengiriman pesanan
PATCH	/api/v1/shipping/:id/assign	JWT: admin	Tugaskan kurir ke manifest pengiriman
PATCH	/api/v1/shipping/:id/status	JWT: kurir	Update status: PROCESSED/SHIPPED/DELIVERED
GET	/api/v1/courier/shipments	JWT: kurir	Daftar manifest yang ditugaskan ke kurir ini

3.6.3 Algoritma Update Status (Anti-IDOR)

60. Autentikasi: Validasi JWT, pastikan role = kurir.
61. Ekstrak id_pengiriman dari URL parameter dan id_user (kurir) dari Gin Context.
62. Validasi Manifest: `SELECT * FROM orders WHERE id = ? AND courier_id = ?` — klausa `AND courier_id` kritis untuk anti-IDOR.
63. Jika nil: return HTTP 403 Forbidden. Catat di `audit_log`.
64. Validasi enum: Pastikan `status_baru` $\in$ {PROCESSED, SHIPPED, DELIVERED}.
65. Validasi transisi: Status hanya boleh maju secara linear (tidak boleh dari DELIVERED ke SHIPPED).
66. `UPDATE orders SET status = ?, shipped_at / delivered_at = NOW() WHERE id = ?`.
67. Return HTTP 200 OK dengan data pengiriman terbaru.

BAB IV — DESAIN BASIS DATA

4.1 Arsitektur Data

Basis data menggunakan PostgreSQL v15 dengan konfigurasi berikut untuk mendukung performa tinggi dan konsistensi data e-commerce:

- Engine: PostgreSQL 15 (mendukung ACID penuh, MVCC, row-level locking)
- Character Set: UTF-8 (mendukung karakter Indonesia dan emoji)
- Isolation Level: Read Committed (default) dengan peningkatan ke Serializable untuk transaksi checkout
- Indexing Strategy: B-Tree Index pada kolom yang sering di-query (email, id_user FK, status)
- Migration Tool: GORM AutoMigrate dengan version-controlled migration files

4.2 Skema Tabel Fisik

4.2.1 Tabel users

Menyimpan profil dan kredensial seluruh aktor sistem. Kolom password hanya menyimpan hash bcrypt — plaintext tidak pernah tersimpan di database.

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
id	VARCHAR(36)	PK	NO	UUIDv4, auto-generate saat insert
name	VARCHAR(100)	-	NO	Nama lengkap pengguna
email	VARCHAR(100)	UNIQUE	NO	B-Tree Index, kunci pencarian login
password	VARCHAR(255)	-	NO	Bcrypt hash (cost 14), min 60 char output
role	VARCHAR(20)	CHECK	NO	Enum: pembeli admin supplier kurir
created_at	TIMESTAMP	-	NO	Auto-set saat INSERT
updated_at	TIMESTAMP	-	NO	Auto-update saat UPDATE
deleted_at	TIMESTAMP	-	YES	Soft delete — NULL = aktif

Index: CREATE UNIQUE INDEX idx_users_email ON users(email); CREATE INDEX idx_users_role ON users(role);

4.2.2 Tabel products (produk)

Etalase komersial platform. Kolom stock dilindungi CHECK CONSTRAINT untuk mencegah nilai negatif, dan kolom user_id mengikat setiap produk ke satu Supplier.

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
id	VARCHAR(36)	PK	NO	UUIDv4
user_id	VARCHAR(36)	FK → users.id	NO	Pemilik produk (Supplier)
category_id	VARCHAR(36)	FK → categories.id	YES	Kategori produk sayuran
name	VARCHAR(150)	-	NO	Nama komoditas sayuran

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
description	TEXT	-	YES	Deskripsi detail produk
price	NUMERIC(15,2)	CHECK > 0	NO	Harga jual, 2 desimal presisi
stock	INTEGER	CHECK >= 0	NO	Stok aktual; CHECK mencegah minus
image_url	TEXT	-	YES	URL gambar di object storage
created_at / updated_at / deleted_at	TIMESTAMP	-	YES/NO	Audit trail & soft delete

Index: CREATE INDEX idx_products_user_id ON products(user_id); CREATE INDEX idx_products_category_id ON products(category_id); CREATE INDEX idx_products_stock ON products(stock) WHERE stock > 0;

4.2.3 Tabel carts (keranjang)

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
id	VARCHAR(36)	PK	NO	UUIDv4
user_id	VARCHAR(36)	FK → users.id	NO	Pemilik keranjang (Pembeli)
product_id	VARCHAR(36)	FK → products.id	NO	Produk yang dipilih
quantity	INTEGER	CHECK >= 1	NO	Minimum 1 unit
created_at / updated_at	TIMESTAMP	-	NO	Audit trail

Unique Constraint: UNIQUE(user_id, product_id) — satu produk hanya muncul satu kali per keranjang (update quantity jika sudah ada).

4.2.4 Tabel orders (Tabel Konsolidasi Pesanan + Pembayaran + Pengiriman)

Berdasarkan keputusan desain aktual implementasi, tabel orders mengkonsolidasikan data pesanan, pembayaran, dan pengiriman dalam satu tabel untuk menyederhanakan query dan mengurangi JOIN overhead.

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
id	VARCHAR(36)	PK	NO	UUIDv4, digunakan sebagai order_id Midtrans
user_id	VARCHAR(36)	FK → users.id	NO	Pembeli yang melakukan pesanan
total_price	NUMERIC(15,2)	-	NO	Total tagihan dihitung server-side
status	VARCHAR(50)	CHECK (enum)	NO	PENDING PAID PROCESSED SHIPPED DELIVERED
payment_token	VARCHAR(255)	-	YES	Snap Token dari Midtrans API
payment_url	TEXT	-	YES	URL redirect halaman pembayaran Midtrans
payment_method	VARCHAR(50)	-	YES	Diisi saat Webhook: gopay bca_va dll
paid_at	TIMESTAMP	-	YES	Diisi saat status → PAID via Webhook

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
courier_id	VARCHAR(36)	FK → users.id	YES	Kurir yang ditugaskan (diisi Admin)
shipping_address	TEXT	-	YES	Alamat pengiriman pembeli
voucher_id	VARCHAR(36)	FK → vouchers.id	YES	Kode diskon yang digunakan (nullable)
discount_amount	NUMERIC(15,2)	-	NO	Nilai diskon (0 jika tidak pakai voucher)
shipped_at	TIMESTAMP	-	YES	Timestamp saat status → SHIPPED
delivered_at	TIMESTAMP	-	YES	Timestamp saat status → DELIVERED
created_at / updated_at	TIMESTAMP	-	NO	Audit trail

4.2.5 Tabel order_items

Nama Kolom	Tipe Data	Constraint	Null	Keterangan
id	VARCHAR(36)	PK	NO	UUIDv4
order_id	VARCHAR(36)	FK → orders.id	NO	Pesanan induk (CASCADE DELETE)
product_id	VARCHAR(36)	FK → products.id RESTRICT	NO	Produk yang dipesan
quantity	INTEGER	CHECK >= 1	NO	Jumlah unit yang dipesan
price_at_purchase	NUMERIC(15,2)	-	NO	Snapshot harga saat checkout (freeze price)
subtotal	NUMERIC(15,2)	-	NO	quantity × price_at_purchase

4.3 Tabel Tambahan (Bonus Implementasi)

Tabel-tabel berikut merupakan implementasi beyond-SKPL yang memperkaya ekosistem platform:

Nama Tabel	Deskripsi & Kolom Utama
categories	Kategori produk: id, name, description, image_url. FK dari products.category_id.
product_variants	Varian produk (ukuran/satuan): id, product_id (FK), variant_name, price, stock.
vouchers	Kode diskon: id, code (UNIQUE), amount, min_order, max_use, expired_at, applied_id (FK users).
wishlists	Daftar keinginan pembeli: id, user_id (FK), product_id (FK). UNIQUE(user_id, product_id).
reviews	Ulasan produk: id, product_id (FK), user_id (FK), rating (1-5), comment, created_at.
disputes	Sengketa/komplain: id, order_id (FK), user_id (FK), reason (TEXT), status, created_at.

Nama Tabel	Deskripsi & Kolom Utama
audit_logs	Log perubahan entitas: id, entity_type, entity_id, action (CREATE/UPDATE/DELETE), actor_id (FK), created_at.

4.4 Strategi Indexing

Index dirancang untuk mengoptimalkan query yang paling sering dieksekusi berdasarkan pola akses aplikasi:

Tabel.Kolom	Tipe Index	Query yang Dioptimalkan	Alasan
users.email	UNIQUE B-Tree	SELECT WHERE email = ?	Login — eksekusi jutaan kali/hari
products.user_id	B-Tree	SELECT WHERE user_id = ?	Tampilkan produk per Supplier
orders.user_id	B-Tree	SELECT WHERE user_id = ?	Riwayat pesanan per Pembeli
orders.status	B-Tree	SELECT WHERE status = ?	Filter pesanan per status (Admin)
orders.courier_id	B-Tree	SELECT WHERE courier_id = ?	Manifest kurir aktif
order_items.order_id	B-Tree	SELECT WHERE order_id = ?	Detail item per pesanan
carts(user_id, product_id)	UNIQUE Composite	SELECT WHERE user_id=? AND product_id=?	Cegah duplikat & percepat lookup

BAB V — DESAIN KONTRAK API (API CONTRACT)

5.1 Konvensi Global API

Seluruh endpoint mengikuti konvensi berikut untuk konsistensi dan kemudahan integrasi frontend:

- Base URL: `https://api.umkm-sayuran.com/api/v1`
- Format Data: Content-Type: `application/json` untuk semua request dan response
- Autentikasi: Authorization: Bearer `<JWT_TOKEN>` pada header setiap request terproteksi
- Versioning: URL-based versioning (`/api/v1/`) untuk backward compatibility
- Pagination: Query params: `?page=1&limit=20&sort=created_at&order=desc`
- Error Format: Selalu `{status: 'error', message: string, errors?: object}` untuk konsistensi error handling di frontend

5.2 Standar Format Response

Response sukses menggunakan format envelope JSON:

Field	Tipe	Keterangan
status	string	'success' 'error' — indikator utama keberhasilan
message	string	Pesan deskriptif untuk logging dan debugging
data	object array	Payload data utama (null jika error)
errors	object	Detail validasi error per-field (opsional, error 400 saja)
meta	object	Informasi pagination: {page, limit, total, total_pages}

5.3 Standar HTTP Status Code

HTTP Code	Kondisi Penggunaan
200 OK	Operasi GET, PUT, PATCH, DELETE berhasil
201 Created	Operasi POST berhasil membuat resource baru (register, checkout, buat produk)
400 Bad Request	Validasi input gagal (payload tidak valid, stok kurang, format salah)
401 Unauthorized	Token tidak ada, expired, atau signature invalid
403 Forbidden	Autentikasi berhasil tapi tidak ber-otoritas (IDOR, role salah)
404 Not Found	Resource yang diminta tidak ditemukan di database
409 Conflict	Konflik data (email sudah terdaftar, state transition tidak valid)
422 Unprocessable Entity	Request valid secara syntax tapi gagal secara semantik bisnis
500 Internal Server Error	Error tidak terduga di server — tidak expose detail internal
502 Bad Gateway	Kegagalan koneksi ke layanan eksternal (Midtrans, Shipping API)

5.4 Ringkasan Semua Endpoint

Method	Endpoint	Auth / Role	Deskripsi Singkat
AUTENTIKASI			
POST	/api/v1/auth/register	Publik	Registrasi akun pembeli
POST	/api/v1/auth/login	Publik	Login semua aktor, return JWT
GET	/api/v1/auth/me	JWT	Profil pengguna aktif
PRODUK & KATALOG			
GET	/api/v1/products	Publik	Daftar produk (filter, sort, pagination)
GET	/api/v1/products/:id	Publik	Detail produk
POST	/api/v1/products	JWT: supplier	Buat produk baru
PUT	/api/v1/products/:id	JWT: supplier	Update produk (IDOR protected)
DELETE	/api/v1/products/:id	JWT: supplier/admin	Hapus produk (IDOR protected)
KERANJANG BELANJA			
GET	/api/v1/shopping-carts	JWT: pembeli	Isi keranjang aktif
POST	/api/v1/shopping-carts	JWT: pembeli	Tambah/update item keranjang
DELETE	/api/v1/shopping-carts/:id	JWT: pembeli	Hapus satu item
PESANAN			
POST	/api/v1/orders/checkout	JWT: pembeli	Checkout ACID, return snap_token
GET	/api/v1/orders	JWT: pembeli	Riwayat pesanan pembeli
GET	/api/v1/orders/:id	JWT: pembeli/admin	Detail pesanan
PATCH	/api/v1/orders/:id/cancel	JWT: pembeli/admin	Batalkan pesanan (PENDING only)
PEMBAYARAN			
POST	/api/v1/payments/webhook	Signature SHA-512	Notifikasi Midtrans (publik dengan signature)
GET	/api/v1/payments/:order_id/status	JWT: pembeli	Cek status pembayaran
PENGIRIMAN			
GET	/api/v1/shipping/:id	JWT: pembeli/kurir	Lacak status pengiriman
PATCH	/api/v1/shipping/:id/status	JWT: kurir	Update status kirim (IDOR protected)
PATCH	/api/v1/shipping/:id/assign	JWT: admin	Tugaskan kurir ke manifest

BAB VI — DESAIN KEAMANAN

6.1 Model Ancaman dan Mitigasi

Berdasarkan analisis OWASP Top 10 dan kebutuhan khusus platform e-commerce multi-aktor, berikut adalah pemetaan ancaman dan strategi mitigasi yang diimplementasikan:

ID Ancaman	Deskripsi Ancaman	Vektor Serangan	Mitigasi
THREAT-01	SQL Injection	Input berbahaya pada query parameter atau payload JSON	GORM parameterized query wajib; tidak ada raw SQL kecuali dengan placeholder (?)
THREAT-02	Broken Authentication	Token palsu, brute-force password, session hijacking	JWT HMAC-SHA256, bcrypt cost 14, token TTL 24 jam, generic error message
THREAT-03	IDOR (Broken Access Control)	Manipulasi ID di URL untuk akses data aktor lain	Query WHERE id = ? AND owner_id = ? wajib di semua operasi mutasi
THREAT-04	Webhook Forgery	Pihak ketiga palsukan notifikasi pembayaran	Validasi SHA-512 signature sebelum proses apapun; tolak jika tidak match
THREAT-05	Overselling / Race Condition	Concurrent checkout produk stok terbatas	SELECT ... FOR UPDATE + ACID transaction + CHECK constraint stock >= 0
THREAT-06	XSS / Injection via JSON	Script berbahaya di field teks yang di-render frontend	Input sanitization di level handler; output escaping di React.js (dangerouslySetInnerHTML dilarang)
THREAT-07	Privilege Escalation	Pembeli mencoba akses endpoint admin/supplier	RoleMiddleware wajib pada semua route group; role diambil dari JWT claim, bukan request body
THREAT-08	Data Exposure	Database server dapat diakses dari internet	Database di private VPC; tidak ada port database terbuka ke publik; koneksi hanya dari App Server

6.2 Desain Lapisan Keamanan Middleware

Middleware dieksekusi sebagai rantai sebelum setiap request mencapai handler bisnis. Urutan eksekusi penting karena setiap middleware dapat menghentikan rantai (short-circuit):

68. RecoveryMiddleware — paling luar; menangkap panic runtime dan mencegah server crash. Return HTTP 500.
69. LoggerMiddleware — merekam method, path, latency, IP, dan status code untuk audit trail.
70. CORSMiddleware — validasi Origin header; hanya izinkan domain frontend yang terdaftar.

71. AuthMiddleware — validasi JWT Bearer token; inject id_user dan role ke Gin Context.
72. RoleMiddleware — validasi role dari Context sesuai yang diizinkan endpoint (wrapper per route group).
73. Handler bisnis — baru dieksekusi jika seluruh middleware di atas lolos.

6.3 Strategi Pengelolaan Secret

Semua nilai sensitif dikelola melalui environment variables dan tidak boleh di-hardcode dalam kode sumber:

- DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME — koneksi database
- JWT_SECRET_KEY — kunci penandatanganan JWT (min 256-bit random string)
- MIDTRANS_SERVER_KEY — autentikasi API Midtrans
- MIDTRANS_CLIENT_KEY — digunakan frontend untuk Snap.js
- APP_ENV — environment flag (development/staging/production) untuk toggle fitur debug

Pada lingkungan produksi, secrets dikelola menggunakan layanan seperti AWS Secrets Manager, HashiCorp Vault, atau Kubernetes Secrets — tidak pernah dari .env file yang di-commit ke repositori.

BAB VII — MATRIKS RANCANGAN PENGUJIAN (SQA)

7.1 Strategi Pengujian

Pengujian sistem mengikuti piramida pengujian yang memastikan setiap lapisan teruji secara memadai:

- Unit Test (70%) — Setiap fungsi usecase dan repository diuji secara terisolasi menggunakan mock/stub.
- Integration Test (20%) — Pengujian interaksi antar komponen: handler + usecase + repository + database nyata (test container).
- End-to-End Test (10%) — Skenario pengguna lengkap dari login hingga penerimaan barang menggunakan Postman Collection / k6.
- Security / Penetration Test — Skenario khusus untuk IDOR, SQL Injection, dan Webhook Forgery.
- Load Test — k6 script untuk mensimulasikan 500+ RPS concurrent checkout.

7.2 Matriks Skenario Pengujian Fungsional

Kode	Modul	Skenario	Expected Result	Pass Criteria
AUTH-01	Autentikasi	Login berhasil (email & password valid)	HTTP 200 + JWT token	Status 200, ada field token & role
AUTH-02	Autentikasi	Login gagal — password salah	HTTP 401 Unauthorized	Status 401, tidak ada token
AUTH-03	Autentikasi	Login gagal — email tidak ada	HTTP 401 Unauthorized	Status 401, pesan generik
AUTH-04	Autentikasi	Akses endpoint terproteksi tanpa token	HTTP 401 dari AuthMiddleware	Status 401, request tidak ke handler
AUTH-05	Autentikasi	Token sudah expired (> 24 jam)	HTTP 401, pesan 'token expired'	Status 401, tidak ada data
CAT-01	Katalog	Supplier update produk miliknya	HTTP 200 + data terbaru	Status 200, stok & harga terupdate
CAT-02	Katalog	Supplier update produk milik Supplier lain (IDOR)	HTTP 403 Forbidden	Status 403, data tidak berubah
CAT-03	Katalog	Buat produk baru (Supplier)	HTTP 201 Created	Status 201, ada id_produk baru
CAT-04	Katalog	Hapus produk milik sendiri	HTTP 200 OK	Produk soft-deleted di DB
CHK-01	Checkout	Checkout berhasil (stok cukup)	HTTP 201 + snap_token	Status 201, stok berkurang, keranjang kosong
CHK-02	Checkout	Checkout gagal — stok tidak cukup	HTTP 400 + detail stok	Status 400, stok tidak berubah, pesanan tidak dibuat
CHK-03	Checkout	Race condition: 2 pembeli checkout bersamaan (stok = 1)	Satu 201, satu 400	Tidak ada overselling (stok >= 0)

Kode	Modul	Skenario	Expected Result	Pass Criteria
CHK-04	Checkout	Checkout tanpa login	HTTP 401 Unauthorized	Status 401, tidak ada pesanan dibuat
PAY-01	Pembayaran	Webhook valid dari Midtrans	HTTP 200, status → PAID	Status 200, status pesanan = PAID
PAY-02	Pembayaran	Webhook signature palsu	HTTP 403 Forbidden	Status 403, status pesanan tetap PENDING
PAY-03	Pembayaran	Webhook duplikat (pesanan sudah PAID)	HTTP 200 tanpa duplikasi	Status 200, tidak ada record ganda
SHP-01	Pengiriman	Kurir update status manifest miliknya	HTTP 200 + status baru	Status 200, status_kirim berubah
SHP-02	Pengiriman	Kurir update manifest milik Kurir lain (IDOR)	HTTP 403 Forbidden	Status 403, manifest tidak berubah
SHP-03	Pengiriman	Update dengan status di luar enum	HTTP 400 Bad Request	Status 400, status tidak berubah

7.3 Skenario Pengujian Non-Fungsional

Kode	Kategori	Skenario	Target / Threshold	Tool
NFR-01	Performa	Latency endpoint standar (GET products)	P95 < 200ms pada 100 concurrent	k6, Grafana
NFR-02	Skalabilitas	Load test checkout endpoint	Min 500 RPS, error rate < 1%	k6 dengan ramp-up scenario
NFR-03	Skalabilitas	Stress test concurrent checkout (race condition)	Stok tidak pernah < 0	k6 concurrent virtual users
NFR-04	Keamanan	SQL Injection pada semua input field	Semua query ter-parameterize	OWASP ZAP, manual testing
NFR-05	Keamanan	Brute-force simulasi 1000 login/menit	Response time tetap > 200ms	ab (Apache Benchmark)
NFR-06	Ketersediaan	Uptime monitoring 30 hari	Uptime >= 99.5% (< 3.6 jam downtime/bulan)	UptimeRobot / Datadog
NFR-07	Keamanan	IDOR mass scan (random UUID)	100% mendapat 403/404	Script Python randomized ID testing

BAB VIII — REVIEW DAN EVALUASI SKPL-UMKM-003

8.1 Ringkasan Penilaian

Dokumen SKPL-UMKM-003 telah diulas secara menyeluruh berdasarkan standar IEEE 830-1998 untuk Software Requirements Specifications. Secara keseluruhan, dokumen ini menunjukkan kualitas yang melampaui standar tugas akademis — terutama dalam kedalaman spesifikasi teknis, pemahaman keamanan sistem, dan perhatian pada aspek SQA.

Aspek Evaluasi	Nilai (1-5)	Keterangan
Kelengkapan (Completeness)	4/5	5 modul utama terdokumentasi lengkap dengan use case, sequence, API spec, dan matriks SQA. Kurang: spesifikasi error recovery dan sistem monitoring.
Konsistensi (Consistency)	4/5	Terminologi konsisten. Terdapat deviasi antara SKPL (tabel terpisah) vs implementasi aktual (tabel konsolidasi orders) yang sudah didokumentasikan dengan baik.
Kejelasan (Clarity)	5/5	Definisi istilah teknis sangat komprehensif. Algoritma proses dideskripsikan step-by-step. Use case dilengkapi precondition, main flow, alt flow, dan postcondition.
Dapat Diuji (Verifiability)	5/5	Setiap use case dilengkapi test case SQA dengan precondition, input, expected result, dan pass criteria yang terukur. Excellent.
Keamanan (Security)	5/5	Penanganan IDOR, SQL Injection, JWT, bcrypt, Webhook SHA-512, dan race condition terdokumentasi dengan sangat baik.
Kesesuaian Standar	4/5	Mengikuti struktur SKPL berbasis objek dengan baik. Beberapa bagian tabel daftar isi menyebut 'Error! Bookmark not defined' (formatting issue).
Inovasi Teknis	5/5	Implementasi beyond-SKPL (8 tabel tambahan: review, wishlist, audit_log, dll) menunjukkan pemahaman mendalam tentang kebutuhan e-commerce nyata.

8.2 Kelebihan Dokumen

- **Kedalaman Teknis Luar Biasa:** Dokumen ini bukan sekadar daftar fitur — ia mendeskripsikan algoritma bisnis konkret (ACID checkout step-by-step), skema SQL aktual, dan kontrak API lengkap dengan contoh JSON payload. Ini jarang ditemukan di dokumen SKPL akademis.
- **Integrasi Keamanan yang Holistik:** Aspek keamanan bukan afterthought — IDOR protection, bcrypt cost 14, JWT stateless, Webhook signature SHA-512, dan row-level locking diintegrasikan ke dalam setiap modul secara organik.
- **Dokumentasi Deviasi Implementasi:** Tim dengan jujur mendokumentasikan perbedaan antara rancangan awal SKPL dengan implementasi aktual (konsolidasi tabel pembayaran dan pengiriman ke orders), lengkap dengan alasan desain.

- Matriks SQA yang Dapat Langsung Digunakan: Test case dilengkapi dengan precondition spesifik, input yang tepat (termasuk URL dan JSON payload), expected result HTTP code, dan pass criteria yang terukur — dapat langsung dieksekusi tim QA.
- Pemodelan Arsitektur Multi-Layer: Sequence diagram mencakup semua lapisan dari frontend hingga database (Midtrans API → Middleware → Handler → Usecase → Repository → PostgreSQL) memberikan visibilitas penuh aliran data.

8.3 Area Perbaikan

8.3.1 Minor Issues (Mudah Diperbaiki)

- Formatting: Beberapa entri di Daftar Isi menampilkan 'Error! Bookmark not defined' — ini indikasi cross-reference Word yang rusak. Perlu regenerasi field ToC.
- Penomoran Konsistensi: Subbab 2.3.3 Data Model Fisik dan 3.1.5/3.1.6 Tabel Pembayaran/Pengiriman di Daftar Isi tidak terisi nomor halaman — kemungkinan karena posisi di dokumen yang bergeser.
- Gambar Sequence Diagram: Sequence diagram (hal. 17) terlalu kecil untuk dibaca secara detail. Disarankan dipecah menjadi 2-3 halaman diagram terpisah dengan resolusi lebih tinggi.

8.3.2 Medium Issues (Perlu Penambahan)

- Spesifikasi Error Recovery: Tidak ada dokumentasi tentang apa yang terjadi jika server Golang crash di tengah transaksi checkout yang sedang berjalan, atau jika koneksi database timeout. Perlu strategi recovery dan compensating transactions.
- Rate Limiting: Dokumen menyebut perlindungan brute-force via bcrypt, namun tidak ada mekanisme rate limiting di level HTTP (misalnya: max 10 request/menit per IP ke endpoint login). Ini celah keamanan yang perlu diaddress.
- API Versioning Strategy: Dokumen menetapkan /api/v1/ namun tidak menjelaskan strategi migrasi ke v2, backward compatibility policy, dan deprecation timeline.
- Spesifikasi Pagination: Endpoint GET /api/v1/products disebutkan mendukung filter dan sort, namun format query parameter dan struktur response pagination tidak dispesifikasikan secara eksplisit.

8.3.3 Major Issues (Perlu Perhatian Serius)

- Token Refresh Mechanism: JWT dengan TTL 24 jam tanpa refresh token mechanism berarti pengguna akan ter-logout setiap 24 jam dan harus login ulang. Untuk UX e-commerce yang baik, diperlukan refresh token dengan sliding window atau TTL lebih panjang.
- Transaksi Pembatalan (Refund Flow): Dokumen mendefinisikan status CANCELLED namun tidak menjelaskan alur pengembalian stok saat pesanan dibatalkan (apakah stok dikembalikan otomatis? Bagaimana dengan refund ke payment gateway Midtrans?).
- Idempotency Key pada Checkout: Jika pembeli menekan tombol checkout dua kali dengan cepat (double-submit), sistem berpotensi membuat dua pesanan. Perlu idempotency key (misalnya X-Idempotency-Key header) untuk mencegah duplicate order.

8.4 Rekomendasi untuk Versi Selanjutnya

74. Tambahkan endpoint POST `/api/v1/auth/refresh-token` untuk mekanisme refresh JWT.
75. Dokumentasikan secara eksplisit compensating transaction saat pembatalan pesanan: `UPDATE products SET stock = stock + quantity FROM order_items WHERE order_id = ?`.
76. Tambahkan middleware rate limiting (sliding window, 10 req/min per IP) khususnya pada endpoint `/auth/login`.
77. Spesifikasikan format pagination response: `{data: [], meta: {page, limit, total, total_pages}}`.
78. Pertimbangkan implementasi Optimistic Locking sebagai alternatif FOR UPDATE untuk meningkatkan throughput pembacaan katalog di bawah beban tinggi.
79. Tambahkan spesifikasi notifikasi real-time (WebSocket atau Server-Sent Events) untuk update status pesanan ke pembeli secara live.
80. Dokumentasikan strategi database backup: frekuensi backup, retention period, dan prosedur point-in-time recovery.

BAB IX — LAMPIRAN

Lampiran A: Pemetaan Kebutuhan SKPL ke SDD

ID SKPL	Kebutuhan	Bab SDD	Komponen Desain
UC-01	Login Pengguna (JWT)	Bab III §3.1	AuthModule, AuthMiddleware, JWT Utility
UC-02	Checkout ACID	Bab III §3.4, Bab IV §4.2.4	OrderModule, ACID Transaction, Row-Lock
UC-03	Webhook Midtrans	Bab III §3.5	PaymentModule, SHA-512 Validator
UC-04	Manajemen Produk Supplier	Bab III §3.2	ProductModule, IDOR Ownership Validation
UC-05	Update Pengiriman Kurir	Bab III §3.6	ShippingModule, Manifest IDOR Protection
NFR-1	Performa < 500ms	Bab IV §4.4, Bab VII §7.3	B-Tree Indexing, GORM Query Optimization
NFR-2	500 RPS	Bab II §2.2, Bab VII §7.3	Golang Goroutines, MVCC PostgreSQL, Docker Scaling
NFR-4	Keamanan OWASP	Bab VI	Seluruh lapisan: GORM, JWT, bcrypt, RBAC, IDOR protection
NFR-5	Konsistensi Data (no oversell)	Bab III §3.4.3	ACID + SELECT FOR UPDATE + CHECK CONSTRAINT

Lampiran B: Daftar Perubahan Dokumen

Versi	Tanggal	Penulis	Deskripsi Perubahan
1.0 — Draft	April 2026	Kelompok 3	Dokumen SDD awal — dibuat berdasarkan SKPL-UMKM-003 Maret 2026

— Akhir Dokumen SDD-UMKM-003 —